Programación II

Comisión 4

Trabajo Final Integrador (Parte 2)

Alumnos

Franco Garré

Román Pereyra

Alvaro Kuschill

Diagrama de clases

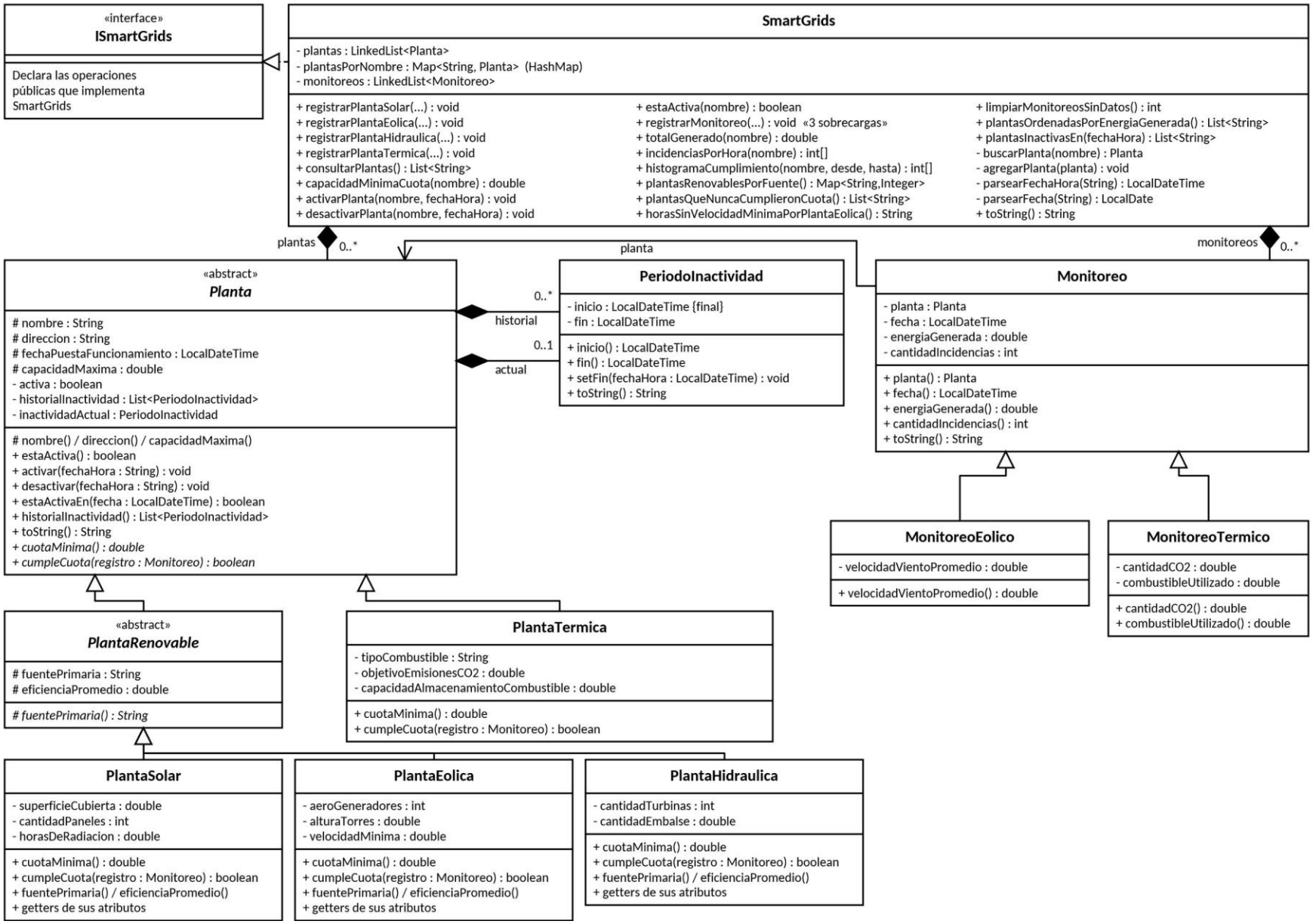

«interface»
ISmartGrids
Declara las operaciones públicas que implementa SmartGrids
SmartGrids
- plantas : LinkedList<Planta>
- plantasPorNombre : Map<String, Planta> (HashMap)
- monitoreos : LinkedList<Monitoreo>
+ registrarPlantaSolar(...) : void
+ registrarPlantaEolica(...) : void
+ registrarPlantaHidraulica(...) : void
+ registrarPlantaTermica(...) : void
+ consultarPlantas() : List<String>
+ capacidadMinimaCuota(nombre) : double
+ activarPlanta(nombre, fechaHora) : void
+ desactivarPlanta(nombre, fechaHora) : void
+ estaActiva(nombre) : boolean
+ registrarMonitoreo(...) : void «3 sobrecargas»
+ totalGenerado(nombre) : double
+ incidenciasPorHora(nombre) : int[]
+ histogramaCumplimiento(nombre, desde, hasta) : int[]
+ plantasRenovablesPorFuente() : Map<String,Integer>
+ plantasQueNuncaCumplieronCuota() : List<String>
+ horasSinVelocidadMinimaPorPlantaEolica() : String
+ limpiarMonitoreosSinDatos() : int
+ plantasOrdenadasPorEnergiaGenerada() : List<String>
+ plantasInactivasEn(fechaHora) : List<String>
- buscarPlanta(nombre) : Planta
- agregarPlanta(planta) : void
- parsearFechaHora(String) : LocalDateTime
- parsearFecha(String) : LocalDate
+ toString() : String
plantas
0..*
planta
monitoreos
0..*
«abstract»
Planta
nombre : String
direccion : String
fechaPuestaFuncionamiento : LocalDateTime
capacidadMaxima : double
- activa : boolean
- historialInactividad : List<PeriodoInactividad>
- inactividadActual : PeriodoInactividad
nombre() / direccion() / capacidadMaxima()
+ estaActiva() : boolean
+ activar(fechaHora : String) : void
+ desactivar(fechaHora : String) : void
+ estaActivaEn(fecha : LocalDateTime) : boolean
+ historialInactividad() : List<PeriodoInactividad>
+ toString() : String
+ cuotaMinima() : double
+ cumpleCuota(registro : Monitoreo) : boolean
0..*
historial
0..1
actual
PeriodoInactividad
- inicio : LocalDateTime {final}
- fin : LocalDateTime
+ inicio() : LocalDateTime
+ fin() : LocalDateTime
+ setFin(fechaHora : LocalDateTime) : void
+ toString() : String
Monitoreo
- planta : Planta
- fecha : LocalDateTime
- energiaGenerada : double
- cantidadIncidencias : int
+ planta() : Planta
+ fecha() : LocalDateTime
+ energiaGenerada() : double
+ cantidadIncidencias() : int
+ toString() : String
MonitoreoEolico
- velocidadVientoPromedio : double
+ velocidadVientoPromedio() : double
MonitoreoTermico
- cantidadCO2 : double
- combustibleUtilizado : double
+ cantidadCO2() : double
+ combustibleUtilizado() : double
«abstract»
PlantaRenovable
fuentePrimaria : String
eficienciaPromedio : double
fuentePrimaria() : String
PlantaTermica
- tipoCombustible : String
- objetivoEmisionesCO2 : double
- capacidadAlmacenamientoCombustible : double
+ cuotaMinima() : double
+ cumpleCuota(registro : Monitoreo) : boolean
PlantaSolar
- superficieCubierta : double
- cantidadPaneles : int
- horasDeRadiacion : double
+ cuotaMinima() : double
+ cumpleCuota(registro : Monitoreo) : boolean
+ fuentePrimaria() / eficienciaPromedio()
+ getters de sus atributos
PlantaEolica
- aeroGeneradores : int
- alturaTorres : double
- velocidadMinima : double
+ cuotaMinima() : double
+ cumpleCuota(registro : Monitoreo) : boolean
+ fuentePrimaria() / eficienciaPromedio()
+ getters de sus atributos
PlantaHidraulica
- cantidadTurbinas : int
- cantidadEmbalse : double
+ cuotaMinima() : double
+ cumpleCuota(registro : Monitoreo) : boolean
+ fuentePrimaria() / eficienciaPromedio()
+ getters de sus atributos

Diagrama de clases actualizado a la implementación. Las firmas completas de SmartGrids están en ISmartGrids; "(...)" abrevia listas largas de parámetros.

Análisis del problema

Esta segunda parte es la implementación del diseño que presentamos en la primera, así que mantuvimos la misma idea de fondo y ajustamos lo que hizo falta al pasar a código. Más abajo hay una sección con todos los cambios y el motivo de cada uno.

Lo primero que vimos en el enunciado es que más adelante van a aparecer otros tipos de planta. Por lo tanto, buscamos un diseño donde agregar una planta nueva no nos obligue a meter mano en el código. Queremos que cuando cambia de un tipo a otro lo resuelva cada planta por su cuenta, y que el sistema trabaje con "plantas" sin fijarse de qué tipo es cada una.

Las cuatro plantas comparten los mismos datos (nombre, dirección, capacidad máxima, estado y fecha de puesta en funcionamiento), así que diseñamos una clase abstracta **Planta** con todo eso. Después vimos que la solar, eólica e hidráulica tienen otra cosa en común: las tres son renovables, y las tres tienen fuente primaria y eficiencia promedio. Para no repetir eso tres veces, creamos la clase **PlantaRenovable**, que hereda de Planta, y de ahí heredan las tres. La térmica no es renovable, así que hereda directo de Planta. Esto además nos viene bien para contar las renovables por fuente, porque alcanza con mirar las que son PlantaRenovable.

Lo que más cambia de un tipo a otro es la cuota de generación. En Planta dejamos **cuotaMinima()** como método abstracto y cada subclase lo calcula a su manera: la solar usa las horas de radiación, la eólica y la hidráulica usan capacidad por eficiencia, y la térmica es la mitad de la capacidad. **cumpleCuota(registro)** recibe un monitoreo y compara lo que generó contra la cuota mínima. En la térmica, además, se fija que el CO2 no llegue al objetivo, y como necesita ese dato exige que el monitoreo sea un MonitoreoTermico.

Con los monitoreos hicimos algo parecido. **Monitoreo** tiene lo que se repite siempre (planta, fecha y hora, energía generada e incidencias). **MonitoreoTermico** le suma el CO2 y el combustible, y **MonitoreoEolico** la velocidad promedio del viento. La solar y la hidráulica no tienen datos extra, así que usan Monitoreo directamente. Si mañana hay que guardar algo nuevo de esas plantas, se crea la subclase en ese momento. A diferencia de la primera parte, ahora el monitoreo sí conoce su planta, y la hora no es un atributo aparte: sale de la fecha y hora (LocalDateTime).

Para guardar los datos, utilizamos una combinación de tipos de estructuras. **SmartGrids** guarda las plantas en una LinkedList (para respetar el orden de registro al consultarlas) y, al mismo tiempo, en un HashMap<String, Planta> con el nombre como clave. Así encontramos una planta por nombre directo, sin recorrer nada, y como el HashMap usa una tabla Hash: buscar, insertar y modificar cuesta O(1) en promedio. Los monitoreos de todas las plantas están en una única LinkedList de SmartGrids.

Sobre el estado, el enunciado dice que solo las plantas activas mandan y reciben datos de monitoreo, y que cada planta tiene que recordar todos los momentos en que estuvo inactiva. Por eso cada planta guarda su estado en un booleano, una lista con los períodos de inactividad ya cerrados y, si está inactiva en este momento, el período abierto. Cada período es un objeto **PeriodoInactividad** con un inicio y un fin. Con eso, activar y desactivar siguen siendo O(1), y **estaActivaEn(fecha)** permite saber si una planta estaba funcionando en cualquier momento del pasado. Lo usamos para rechazar monitoreos de horas en las que la planta estaba inactiva y para el punto 13.

Hay cosas que el enunciado no aclara y tuvimos que suponerlas según nuestro criterio de diseño:

- La eficiencia promedio va entre 0 (sin incluirlo) y 1 (por ejemplo 0,85).
- La cuota es por hora pero las horas de radiación son por día, así que en la solar dividimos por 24: capacidad máxima × eficiencia × horas de radiación / 24.
- Si una planta todavía no tiene monitoreos, consideramos que nunca cumplió la cuota.

- Las fechas se reciben como texto con el formato ISO que usan LocalDate y LocalDateTime (por ejemplo "2026-10-04T13:00:00"), y la hora de un monitoreo es la hora de esa fecha (0 a 23).
- Para una planta, dos monitoreos son “de la misma hora” si coinciden el día y la hora, sin importar los minutos.
- Los monitoreos no se modifican una vez creados, porque no hay ninguna operación que los cambie.
- Pasar una planta a inactiva no borra los monitoreos que ya tenía, porque se cargaron cuando estaba activa.
- Una planta registrada con activa = false (desde el constructor) cuenta como inactiva desde su fecha de puesta en funcionamiento.

Cambios respecto de la primera parte

Al implementar, algunas cosas del diseño de la primera parte cambiaron. Las resumimos acá, con el motivo y lo que nos costó cada decisión.

Tema	Qué cambió	Por qué
Dónde se guardan los registros	Antes cada Planta guardaba un Diccionario<String, Registro>. Ahora SmartGrids guarda una única lista de monitoreos de todas las plantas.	La cátedra agregó funcionalidades que trabajan sobre todos los monitoreos, como limpiar los que no tienen datos (con un solo Iterator) y ordenar las plantas por energía generada. A cambio, la unicidad por planta, fecha y hora ya no sale de una clave y se chequea recorriendo la lista al registrar.
Clases de registro	Registro, RegistroSolar, RegistroHidraulico, RegistroEolico y RegistroTermico pasaron a Monitoreo, MonitoreoEolico y MonitoreoTermico. Monitoreo ya no es abstracto.	Seguimos el vocabulario de la consigna. Como solar e hidráulica no tienen datos propios, usan Monitoreo directamente en vez de subclases vacías.
Planta de un monitoreo	Antes el registro no sabía de qué planta era. Ahora Monitoreo tiene un atributo planta.	Hace falta porque todos los monitoreos están en una lista común.
Fecha y hora	Antes había un tipo Fecha y una hora entera aparte. Ahora se usa LocalDateTime y LocalDate de Java.	Las fechas llegan como texto en el formato de esas clases, y la hora sale de getHour().
Tema	**Qué cambió**	**Por qué**

Plantas en SmartGrids	Antes un Diccionario<String, Planta>. Ahora una LinkedList más un HashMap por nombre.	La lista mantiene el orden de registro y el HashMap mantiene la búsqueda directa, necesaria para que activar, desactivar y consultar el estado sean O(1).
Estado de la planta	Antes solo un booleano. Ahora además guarda el historial de inactividad con objetos PeriodoInactividad (clase nueva).	Es la definición que se agregó en esta segunda parte: cada planta debe conocer todos los momentos en que estuvo inactiva.
Cuota	cumpleCuota(fecha, hora) pasó a cumpleCuota(registro).	Como los monitoreos ya no están dentro de la planta, la planta evalúa el monitoreo que recibe.
Operaciones de SmartGrids	Los nombres y parámetros siguen la interfaz ISmartGrids de la cátedra. Por ejemplo, actualizarEstadoPlanta pasó a activarPlanta y desactivarPlanta (con fecha), registrarMonitoreo tiene tres versiones y el histograma devuelve un arreglo de 24 posiciones.	La interfaz viene dada y no se puede modificar.
Condiciones de los datos	Algunas son más estrictas: capacidades y cantidades mayores que cero, números finitos, textos que no estén vacíos.	Al escribir los constructores vimos que había valores que no tenían sentido.

TADs e IREP

En esta sección armamos cada clase como un TAD. De cada uno ponemos la **representación** (con qué atributos se guarda), el **IREP** con una explicación de por qué pedimos cada condición, y las **operaciones**, diciendo qué necesitan para poder usarse y qué dejan cumplido al terminar.

El IREP (invariante de representación) es lo que se tiene que cumplir siempre: cuando termina el constructor, y antes y después de cada operación pública. Las subclases cumplen el IREP de la clase que heredan más el suyo. En las tablas de operaciones, “Requiere” es lo que tiene que valer antes de llamarla (la precondición) y “Asegura” es lo que queda valiendo cuando termina. Todas las clases tienen su constructor, que pide datos que respeten el IREP, y un método para consultar cada atributo.

TAD SmartGrids

Es el sistema que maneja todas las plantas y todos los monitoreos. Implementa la interfaz ISmartGrids, así que sus operaciones son las que usa el resto del programa.

Representación `plantas` :
`Lista<Planta>`

```
plantasPorNombre : Diccionario<String, Planta> monitoreos
: Lista<Monitoreo>
```

IREP

N°	Condición (siempre debe cumplirse)	Por qué
1	plantas, plantasPorNombre y monitoreos no son nulos.	Sin estas estructuras no anda ninguna operación. Con el sistema vacío existen, solo que sin elementos.
2	Ningún elemento de plantas ni de monitoreos es nulo, y tampoco lo son las claves ni los valores del diccionario.	Cada entrada tiene que ser una planta o un monitoreo de verdad.
3	Para cada entrada (k, p) de plantasPorNombre: k = p.nombre().	La clave es el nombre de la planta. Así buscar por nombre es directo y la clave nunca se desfasa del dato real.
4	plantas y plantasPorNombre tienen exactamente las mismas plantas. No existen dos plantas con el mismo nombre.	La lista y el diccionario se actualizan juntos en agregarPlanta(). La unicidad sale de la condición 3 y de que un diccionario no repite claves.
5	La planta de cada monitoreo está en plantas.	Un monitoreo de una planta que no existe en el sistema no tendría sentido.
6	No existen dos monitoreos de la misma planta con el mismo día y la misma hora.	Hay un solo monitoreo por planta y por hora. Como ya no sale de una clave, lo controla registrarMonitoreo recorriendo la lista.
7	El tipo de cada monitoreo corresponde al tipo de su planta: MonitoreoEolico para PlantaEolica, MonitoreoTermico para PlantaTermica y Monitoreo común para PlantaSolar y PlantaHidraulica.	Las plantas necesitan los datos propios de su tipo (viento, CO2) para calcular la cuota y los puntos 7 y 10.
8	La fecha de cada monitoreo no es anterior a la puesta en funcionamiento de su planta, y la planta estaba activa en ese momento cuando se lo registró.	Es la regla del enunciado: solo las plantas activas mandan datos.
9	Todas las plantas y todos los monitoreos cumplen el IREP de su clase.	SmartGrids los guarda, así que cada uno tiene que estar bien armado.

Operaciones

Operación	Requiere	Asegura

registrarPlantaSolar(nombre, direccion, capacidadMaxima, fechaHora, fuentePrimaria, eficiencia, superficie, paneles, horasRadiacion) : void (y las equivalentes de PlantaEolica, PlantaHidraulica y PlantaTermica)	No hay una planta con ese nombre. fechaHora con formato válido. Los datos cumplen el IREP de la planta que se crea.	En plantas y en plantasPorNombre queda una planta nueva, activa y sin monitoreos, con ese nombre como clave. Las demás plantas no cambian. Es O(1) en promedio.

Operación	**Requiere**	**Asegura**
consultarPlantas() : List<String>	Ninguno.	Devuelve una línea por planta, en orden de registro, con el formato "{NOMBRE} ({ACTIVA|INACTIVA}): {FECHA}". No modifica nada.
capacidadMinimaCuota(nombre) : double	Existe una planta con ese nombre.	Devuelve la cuotaMinima() de esa planta, es decir, lo mínimo que tiene que generar por hora. No modifica nada.
activarPlanta(nombre, fechaHora) : void desactivarPlanta(nombre, fechaHora) : void	Existe una planta con ese nombre. fechaHora con formato válido. Al activar, la fecha no es anterior al inicio de la inactividad actual.	La planta queda activa (o inactiva). Al desactivar se abre un PeriodoInactividad con esa fecha; al activar se cierra y pasa al historial. Si la planta ya estaba en ese estado, no cambia nada. Es O(1).
estaActiva(nombre) : boolean	Existe una planta con ese nombre.	Devuelve el estado actual de la planta. Es O(1).
registrarMonitoreo(nombre, fechaHora, energia, incidencias) : void registrarMonitoreo(nombre, fechaHora, energia, incidencias, velocidadViento) : void registrarMonitoreo(nombre, fechaHora, energia, incidencias, co2, combustible) : void	Existe una planta con ese nombre y está activa. La planta estaba activa en esa fecha y hora, que no es anterior a su puesta en funcionamiento. fechaHora con formato válido. energia, incidencias, velocidad, co2 y combustible ≥ 0 y finitos. La planta es del tipo que corresponde a la versión usada (solar o hidráulica, eólica, térmica). La planta no tiene ya un monitoreo en ese día y esa hora.	monitoreos tiene un monitoreo nuevo, del tipo que corresponde, con esos datos. Los demás monitoreos no cambian. Cuesta O(M) por el chequeo de la hora repetida.

totalGenerado(nombre) : double	Existe una planta con ese nombre.	Devuelve la suma de la energiaGenerada de todos los monitoreos de la planta (0 si no tiene ninguno).
incidenciasPorHora(nombre) : int[]	Existe una planta con ese nombre.	Devuelve un arreglo de 24 posiciones. La posición h es la suma de incidencias de los monitoreos de esa planta cuya hora es h, de cualquier fecha.
histogramaCumplimiento(nombre, fechaDesde, fechaHasta) : int[]	Existe una planta con ese nombre. Las fechas tienen formato válido y desde ≤ hasta.	Devuelve un arreglo de 24 posiciones. La posición h es la cantidad de monitoreos de esa planta con hora h, con fecha entre desde y hasta (inclusive), que cumplen la cuota. Como hay uno por día y hora, es la cantidad de días. Se analiza en la sección de complejidad.
Operación	**Requiere**	**Asegura**
plantasRenovablesPorFuente() : Map<String, Integer>	Ninguno.	Devuelve, ordenado por fuente primaria, cuántas PlantaRenovable hay de cada fuente. Las fuentes sin plantas no figuran.
plantasQueNuncaCumplieronCuota() : List<String>	Ninguno.	Devuelve los nombres de las plantas que no tienen ningún monitoreo que cumpla la cuota. Las que no tienen monitoreos están en la lista.
horasSinVelocidadMinimaPorPlantaEolica() : String	Ninguno.	Devuelve el texto con el formato de la consigna: el título y, por cada PlantaEolica, su velocidad mínima y las horas en las que ningún monitoreo la alcanzó, con la mayor velocidad registrada en esa hora.
limpiarMonitoreosSinDatos() : int	Ninguno.	Elimina todos los monitoreos con energía e incidencias en cero y devuelve cuántos eliminó. Los demás quedan en el mismo orden.
plantasOrdenadasPorEnergiaGenerada() : List<String>	Ninguno.	Devuelve una línea “{NOMBRE}: {TOTAL}” por planta, de mayor a menor energía generada. No modifica las estructuras internas.
plantasInactivasEn(fechaHora) : List<String>	fechaHora con formato válido.	Devuelve los nombres de las plantas para las que estaActivaEn(fecha) es falso.

toString() : String	Ninguno.	Devuelve un texto con el título, una línea por cada planta (su toString()) y una línea por cada monitoreo (su toString()). No modifica nada.

TAD Planta (abstracta)

Tiene lo que comparten todas las plantas. Además de sus datos, guarda su estado y el historial de los períodos en que estuvo inactiva.

Representación `nombre :`
`String direccion :`
`String capacidadMaxima :`
`double`
`fechaPuestaFuncionamiento : LocalDateTime`
`activa : boolean`
`historialInactividad : Lista<PeriodoInactividad>`
`inactividadActual : PeriodoInactividad (null si la planta está activa)`

IREP

N°	Condición (siempre debe cumplirse)	Por qué
1	nombre ≠ null y no está vacío ni en blanco.	Es el dato con el que SmartGrids identifica a la planta.
2	direccion ≠ null y no está vacía ni en blanco.	Toda planta está en algún lado, así que tiene que tener dirección.
3	capacidadMaxima es finita y mayor que 0.	Las cuotas se calculan a partir de ella, y una capacidad nula o negativa no tiene sentido.
4	fechaPuestaFuncionamiento ≠ null.	Es un dato obligatorio, y con él se decide desde cuándo puede tener monitoreos.
5	historialInactividad ≠ null, no tiene elementos nulos y todos sus períodos están cerrados (fin ≠ null).	Solo pasan al historial los períodos que terminaron, cuando la planta se vuelve a activar.
6	activa es verdadero si y solo si inactividadActual es null. Si inactividadActual no es null, es un período abierto (fin = null).	Una planta inactiva tiene siempre un período en curso, y una activa no. Así activar y desactivar son O(1).

Operaciones

Operación	Requiere	Asegura
nombre(), direccion(), capacidadMaxima(), estaActiva()	Ninguno.	Devuelven el valor del atributo. No modifican nada.

activar(fechaHora : String) : void	fechaHora con formato válido. Si la planta está inactiva, la fecha no es anterior al inicio de la inactividad actual.	Si estaba inactiva, queda activa y su período actual, ya cerrado con esa fecha, se agrega al final del historial. Si ya estaba activa, no cambia nada. Es O(1).
desactivar(fechaHora : String) : void	fechaHora con formato válido.	Si estaba activa, queda inactiva con un período abierto que arranca en esa fecha. Si ya estaba inactiva, no cambia nada. Es O(1).
estaActivaEn(fecha : LocalDateTime) : boolean	fecha ≠ null.	Da falso si la fecha es anterior a la puesta en funcionamiento, si cae dentro de un período del historial (desde el inicio hasta el fin, sin incluirlo) o es igual o posterior al inicio de la inactividad actual. En otro caso da verdadero.
historialInactividad() : List<PeriodoInactividad>	Ninguno.	Devuelve una copia no modificable de los períodos cerrados. No cambia la planta.
cuotaMinima() : double (abstracto)	Ninguno.	Devuelve lo mínimo que la planta tiene que generar por hora. Cada subclase tiene su fórmula.
Operación	**Requiere**	**Asegura**
cumpleCuota(registro : Monitoreo) : boolean (abstracto)	registro ≠ null.	Da verdadero si el monitoreo cumple la cuota de esa planta. Cada subclase define cómo.
toString() : String	Ninguno.	Devuelve “{nombre} ({ACTIVA|INACTIVA}): {fechaPuestaFuncionamiento}”, con la fecha con formato yyyy-MMdd'T'HH:mm:ss.

TAD PeriodoInactividad

Es un intervalo en el que una planta estuvo inactiva. Mientras la planta sigue inactiva, el período está abierto.

Representación `inicio : LocalDateTime`
`fin : LocalDateTime (null mientras está abierto)`

IREP

N°	Condición (siempre debe cumplirse)	Por qué
1	inicio ≠ null.	Un período siempre empieza en algún momento.

2	fin es null (período abierto) o bien fin no es anterior a inicio.	Un período no puede terminar antes de empezar.
3	inicio no cambia y fin se asigna una sola vez.	El historial no se puede reescribir: un período cerrado queda fijo.

Operaciones

Operación	Requiere	Asegura
inicio(), fin()	Ninguno.	Devuelven el valor del atributo (fin es null si está abierto). No modifican nada.
setFin(fechaHora : LocalDateTime) : void	fechaHora ≠ null. El período está abierto. fechaHora no es anterior a inicio.	fin = fechaHora. El período queda cerrado y no se puede volver a cerrar.
toString() : String	Ninguno.	Devuelve el inicio y el fin en forma de texto.

TAD PlantaRenovable (abstracta)

Hereda de Planta. Junta lo que comparten la solar, la eólica y la hidráulica.

Representación (además de la de Planta)

```
fuentePrimaria      :      String
eficienciaPromedio : double
```

IREP (además del de Planta)

N°	Condición (siempre debe cumplirse)	Por qué
1	fuentePrimaria ≠ null y no está vacía ni en blanco.	Es el dato por el que se cuentan las renovables, así que no puede faltar.
2	eficienciaPromedio es finita, mayor que 0 y menor o igual que 1.	Es una proporción (por ejemplo 0,85). Fuera de ese rango, la cuota mínima daría menos que cero o más que la capacidad máxima, y no tiene sentido.

Operaciones

Operación	Requiere	Asegura
fuentePrimaria() : String (abstracto), eficienciaPromedio() : double	Ninguno.	Devuelven el valor del atributo. No modifican nada.

TAD PlantaSolar

Hereda de PlantaRenovable. Sus monitoreos son Monitoreo comunes.

Representación (además de la heredada)

```
superficieCubierta    :    double
cantidadPaneles       :       int
horasDeRadiacion : double
```

IREP (además del de PlantaRenovable)

N°	Condición (siempre debe cumplirse)	Por qué
1	superficieCubierta es finita y mayor o igual que 0.	Una superficie no puede ser negativa.
2	cantidadPaneles > 0.	Una planta solar tiene al menos un panel.
3	horasDeRadiacion > 0.	Son las horas de radiación por día. Las usamos para calcular la cuota, y con cero la cuota sería cero.

Operaciones

Operación	Requiere	Asegura
cuotaMinima() : double	Ninguno.	Devuelve capacidadMaxima × eficienciaPromedio × horasDeRadiacion / 24.
cumpleCuota(registro : Monitoreo) : boolean	registro ≠ null.	Da verdadero si registro.energiaGenerada() ≥ cuotaMinima().

TAD PlantaEolica

Hereda de PlantaRenovable. Sus monitoreos son MonitoreoEolico.

Representación (además de la heredada)

```
aeroGeneradores : int
alturaTorres : double
velocidadMinima : double
```

IREP (además del de PlantaRenovable)

N°	Condición (siempre debe cumplirse)	Por qué
1	aeroGeneradores > 0.	Una planta eólica tiene al menos un aerogenerador.
2	alturaTorres es finita y mayor que 0.	Una torre tiene que tener altura. Pedimos mayor que cero porque una torre de altura 0 no existe.
3	velocidadMinima es finita y mayor que 0.	Es la velocidad de viento necesaria para generar. La usamos en el punto 10 para ver qué horas no la alcanzaron.

Operaciones

Operación	Requiere	Asegura
cuotaMinima() : double	Ninguno.	Devuelve capacidadMaxima × eficienciaPromedio.
cumpleCuota(registro : Monitoreo) : boolean	registro ≠ null.	Da verdadero si registro.energiaGenerada() ≥ cuotaMinima().

TAD PlantaHidraulica

Hereda de PlantaRenovable. Sus monitoreos son Monitoreo comunes.

Representación (además de la heredada)

```
cantidadTurbinas : int
cantidadEmbalse : double
```

IREP (además del de PlantaRenovable)

N°	Condición (siempre debe cumplirse)	Por qué
1	cantidadTurbinas > 0.	Una planta hidráulica tiene al menos una turbina.
2	cantidadEmbalse es finita y mayor que 0.	La capacidad del embalse no puede ser nula ni negativa.

Operaciones

Operación	Requiere	Asegura
cuotaMinima() : double	Ninguno.	Devuelve capacidadMaxima × eficienciaPromedio.
cumpleCuota(registro : Monitoreo) : boolean	registro ≠ null.	Da verdadero si registro.energiaGenerada() ≥ cuotaMinima().

TAD PlantaTermica

Hereda directo de Planta porque no es renovable. Sus monitoreos son MonitoreoTermico.

Representación (además de la de Planta)

```
tipoCombustible : String
objetivoEmisionesCO2 : double
capacidadAlmacenamientoCombustible : double
```

IREP (además del de Planta)

N°	Condición (siempre debe cumplirse)	Por qué
1	tipoCombustible ≠ null y no está vacío ni en blanco.	Una planta térmica siempre quema algún combustible.
2	objetivoEmisionesCO2 es finito y mayor o igual que 0.	Es un tope de emisiones por hora, no puede ser negativo. Lo usamos para decidir si cumple la cuota.
3	capacidadAlmacenamientoCombustible es finita y mayor o igual que 0.	Una capacidad de almacenamiento no puede ser negativa.

Operaciones

Operación	Requiere	Asegura
cuotaMinima() : double	Ninguno.	Devuelve capacidadMaxima / 2.

cumpleCuota(registro : Monitoreo) : boolean (redefinido)	registro ≠ null y es un MonitoreoTermico (si no, lanza IllegalArgumentException).	Da verdadero si registro.energiaGenerada() ≥ cuotaMinima() y registro.cantidadCO2() < objetivoEmisionesCO2.

TAD Monitoreo

Es una medición de una hora de una planta. Las solares y las hidráulicas usan esta clase directamente.

Representación `planta : Planta`
`fecha : LocalDateTime`
`energiaGenerada : double`
`cantidadIncidencias : int`

IREP

N°	Condición (siempre debe cumplirse)	Por qué
1	planta ≠ null.	Todos los monitoreos están en una lista común, así que cada uno tiene que saber a qué planta pertenece.
2	fecha ≠ null.	Con la fecha y la hora se ubica al monitoreo en el histograma y se controla que no haya dos en la misma hora.
N°	**Condición (siempre debe cumplirse)**	**Por qué**
3	energiaGenerada es finita y mayor o igual que 0.	Una planta no puede generar energía negativa.
4	cantidadIncidencias ≥ 0.	Es un conteo, no puede ser negativo.

Operaciones

Operación	Requiere	Asegura
planta(), fecha(), energiaGenerada(), cantidadIncidencias()	Ninguno.	Devuelven el valor del atributo. El monitoreo no tiene ninguna operación que lo modifique, así que el IREP se mantiene desde que se crea.
toString() : String	Ninguno.	Devuelve un texto con el nombre de la planta, la fecha, la energía generada y las incidencias.

TAD MonitoreoEolico

Hereda de Monitoreo.

Representación (además de la de Monitoreo) `velocidadVientoPromedio : double` **IREP (además del de Monitoreo)**

N°	Condición (siempre debe cumplirse)	Por qué

1	velocidadVientoPromedio es finita y mayor o igual que 0.	Es una velocidad, no puede ser negativa. Se compara con la velocidad mínima de la planta eólica para ver si esa hora tuvo viento suficiente.

Operaciones

Operación	Requiere	Asegura
velocidadVientoPromedio() : double	Ninguno.	Devuelve el valor del atributo. No modifica nada.

TAD MonitoreoTermico

Hereda de Monitoreo.

Representación (además de la de Monitoreo)

```
cantidadCO2          :          double
combustibleUtilizado : double
```

IREP (además del de Monitoreo)

N°	Condición (siempre debe cumplirse)	Por qué
1	cantidadCO2 es finita y mayor o igual que 0.	Una emisión no puede ser negativa. Se compara con el objetivo de emisiones de la planta térmica.
N°	**Condición (siempre debe cumplirse)**	**Por qué**
2	combustibleUtilizado es finito y mayor o igual que 0.	No se puede usar una cantidad negativa de combustible.

Además, un MonitoreoTermico solo puede estar asociado a una PlantaTermica (y lo mismo con MonitoreoEolico y PlantaEolica). Eso no se puede chequear desde el monitoreo, así que lo cuida SmartGrids: es la condición 7 de su IREP y lo verifica cada versión de registrarMonitoreo.

Operaciones

Operación	Requiere	Asegura
cantidadCO2() : double, combustibleUtilizado() : double	Ninguno.	Devuelven el valor del atributo. No modifican nada.

Herencia, polimorfismo y otras herramientas de Java

La consigna pide explicar dónde usamos cada herramienta de Java. Lo resumimos en esta tabla.

Concepto	Dónde lo usamos	Para qué
Herencia	Planta → PlantaRenovable → PlantaSolar, PlantaEolica, PlantaHidraulica. Planta → PlantaTermica. Monitoreo → MonitoreoEolico, MonitoreoTermico.	Reutilizar los atributos y el comportamiento comunes (activación, historial de inactividad, datos básicos del monitoreo) sin repetir código.

Clases y métodos abstractos	Planta (cuotaMinima() y cumpleCuota() abstractos) y PlantaRenovable (fuentePrimaria() abstracto).	Impedir que se cree una planta "genérica" y obligar a cada tipo concreto a definir su propia cuota.
Polimorfismo	SmartGrids trabaja siempre con variables de tipo Planta. Por ejemplo, en histogramaCumplimiento() y plantasQueNuncaCumplieronCuota() se llama a p.cumpleCuota(m) sin preguntar de qué tipo es la planta.	Que cada planta ejecute su propia versión del cálculo y que agregar un tipo nuevo no obligue a tocar esos métodos.
Sobreescritura	cuotaMinima() y cumpleCuota() en las cuatro plantas concretas; toString() en Planta, PeriodoInactividad, Monitoreo y SmartGrids; los métodos de SmartGrids que implementan ISmartGrids.	Dar a cada subclase su comportamiento específico.
Sobrecarga	registrarMonitoreo() tiene tres versiones en SmartGrids: una para solar e hidráulica, otra para eólica (con velocidad del viento) y otra para térmica (con CO2 y combustible).	Usar el mismo nombre para la misma acción, y que los parámetros determinen el tipo de monitoreo que se crea.
Concepto	**Dónde lo usamos**	**Para qué**
Interfaz	ISmartGrids, que provee la cátedra y no modificamos. SmartGrids la implementa.	Separar lo que el sistema ofrece de cómo está implementado.
StringBuilder	horasSinVelocidadMinimaPorPlantaEolica() (punto 10) y toString() de SmartGrids.	El texto de salida se arma agregando muchos pedazos (títulos, nombres, horas, una línea por planta y por monitoreo). Con StringBuilder no se crea un String nuevo en cada concatenación.
Iteradores y foreach	Foreach en la mayoría de los recorridos (consultarPlantas, totalGenerado, histogramaCumplimiento, etc.). Iterator explícito en limpiarMonitoreosSinDatos() (punto 11).	Recorrer las listas sin usar índices. En el punto 11 hace falta el Iterator porque se borra mientras se recorre, y hacerlo con foreach lanzaría ConcurrentModificationException.

Análisis de complejidad

Punto 3: activar, desactivar y consultar el estado en O(1)

La interfaz pide que estas tres operaciones se resuelvan en O(1). Todas empiezan igual: llaman a buscarPlanta(nombre), que hace un get() sobre el HashMap plantasPorNombre. Un HashMap calcula el hash de la clave (el nombre, de longitud acotada) y accede directamente al bucket, así que el costo esperado es O(1), sin importar cuántas plantas haya registradas.

- **activarPlanta() y desactivarPlanta():** buscarPlanta() O(1) + control de planta inexistente O(1) + Planta.activar() o Planta.desactivar(). Estos últimos parsean la fecha (un String de formato fijo, O(1)), comparan fechas (O(1)), crean o cierran un PeriodoInactividad (O(1)) y, al activar, agregan el período cerrado al final de un ArrayList (O(1) amortizado). Total: O(1) + O(1) + O(1) = O(1).
- **estaActiva():** buscarPlanta() O(1) + lectura del atributo activa O(1) = O(1). No recorre el historial, porque el estado actual está guardado en un booleano.

Como consecuencia, agregarPlanta() también es O(1) en promedio, ya que controlar que el nombre no esté repetido es otro get() sobre el HashMap.

Punto 7: histograma de cumplimiento de cuota por hora

Llamamos **M** a la cantidad total de monitoreos guardados (de todas las plantas). Estas son todas las funciones que intervienen.

```
public int[] histogramaCumplimiento(String nombre,
        String fechaDesde, String fechaHasta) {
    Planta p = buscarPlanta(nombre);
    if (p == null) {
        throw new RuntimeException("La planta no existe o está inactiva.");
    }
    LocalDate desde = parsearFecha(fechaDesde);
LocalDate hasta = parsearFecha(fechaHasta);

    if (desde.isAfter(hasta)) {
throw new RuntimeException(
            "La fecha desde no puede ser posterior a la fecha hasta.");
    }
    int[] histograma = new int[24];

    for (Monitoreo m : monitoreos) {
        LocalDate diaRegistro = m.fecha().toLocalDate();
        if (m.planta().equals(p) && !diaRegistro.isBefore(desde)
                && !diaRegistro.isAfter(hasta) && p.cumpleCuota(m)) {
            int hora = m.fecha().getHour();
            histograma[hora]++;
        }
    }
    return histograma;
}
```

```
private Planta buscarPlanta(String nombre) {
return plantasPorNombre.get(nombre); }
```

```
private LocalDate parsearFecha(String fecha) {
if (fecha == null || fecha.isBlank()) {
throw new IllegalArgumentException(
            "La fecha no puede estar vacía.");
    }
try {
        return LocalDate.parse(fecha);
} catch (DateTimeParseException e) {
throw new IllegalArgumentException(
            "El formato de fecha es inválido. "
            + "Ejemplo válido: 2026-03-01.", e);
    }
}
```

Y las funciones de cuota de las plantas. Las de PlantaEolica y PlantaHidraulica son iguales a las de PlantaSolar, salvo que cuotaMinima() no multiplica ni divide por las horas de radiación:

```
// PlantaSolar
public double cuotaMinima() {
    double cuotaMin = (this.capacidadMaxima * this.eficienciaPromedio
            * this.horasDeRadiacion) / 24;
    return cuotaMin;
}
public boolean cumpleCuota(Monitoreo registro) {
    if (registro == null) {
        throw new IllegalArgumentException(
            "El monitoreo no puede ser null.");
    }
    return registro.energiaGenerada() >= cuotaMinima();
}
```

```
// PlantaTermica
public double cuotaMinima() {
    double cuotaMin = this.capacidadMaxima / 2;
    return cuotaMin;
}
public boolean cumpleCuota(Monitoreo registro) {
    if (registro == null) {
        throw new IllegalArgumentException(
            "El monitoreo no puede ser null.");
    }
    if (!(registro instanceof MonitoreoTermico)) {
throw new IllegalArgumentException(
            "El registro debe ser un monitoreo térmico.");
    }
    MonitoreoTermico registroTermico = (MonitoreoTermico) registro;
return registroTermico.energiaGenerada() >= cuotaMinima()
        && registroTermico.cantidadCO2() < this.objetivoEmisionesCO2;
}
```

Análisis con álgebra de órdenes

Primero analizamos las funciones auxiliares, de adentro hacia afuera.

- **cuotaMinima()** (en cualquiera de las cuatro plantas) hace unas pocas operaciones aritméticas sobre atributos: O(1).
- **cumpleCuota()** verifica que el registro no sea null, llama a energiaGenerada() (un getter) y a cuotaMinima(), y compara. En PlantaTermica además hay un instanceof, un cast y otra comparación. Todo es una cantidad fija de pasos: O(1) + O(1) + O(1) = O(1).
- **parsearFecha()** valida el String y llama a LocalDate.parse sobre un texto de formato fijo (aaaa-mm-dd), que tiene siempre la misma cantidad de caracteres: O(1).
- **buscarPlanta()** es un get() sobre el HashMap: O(1) esperado, independiente de la cantidad de plantas.

Ahora el método principal, *histogramaCumplimiento()*, paso por paso:

Paso	**Orden**
buscarPlanta(nombre)	O(1)

Paso	Orden
Chequeo de planta null	O(1)
Dos llamadas a parsearFecha()	O(1) + O(1) = O(1)
desde.isAfter(hasta)	O(1)
Crear int[24]	O(1) (tamaño fijo)
Foreach sobre monitoreos: M iteraciones, con cuerpo O(1)	M · O(1) = O(M)
return	O(1)

El cuerpo del ciclo es O(1) porque cada iteración hace una cantidad fija de operaciones: toLocalDate(), equals() sobre la planta, dos comparaciones de fechas, cumpleCuota() (que es O(1) como vimos), getHour() y el incremento de una posición del arreglo.

Aplicando la regla de la suma:

$$O(1) + O(1) + O(1) + O(1) + O(1) + O(M) + O(1) = O(M)$$

Por lo tanto, la complejidad temporal lograda es **O(M)**. La complejidad espacial adicional es O(1), porque lo único que se crea es el arreglo de 24 enteros.

Comentarios sobre el resultado

- Recorrimos los monitoreos con foreach, es decir, con el iterador de la lista. Si hubiéramos usado get(i) sobre una LinkedList, cada acceso costaría O(i) y el ciclo completo pasaría a ser $O(M^2)$.
- El costo no depende de la cantidad de días entre las dos fechas. Como filtramos cada monitoreo comparando su fecha con el rango, no hace falta recorrer los días uno por uno (lo que habría sumado un factor D, la cantidad de días del rango).
- M cuenta los monitoreos de todas las plantas, no solo los de la planta consultada, porque todos están en la misma lista de SmartGrids. En la primera parte cada planta guardaba los suyos y el recorrido habría sido proporcional a los de esa planta. Una mejora posible sería volver a esa idea.

Otras opciones que pensamos

Antes de quedarnos con este diseño, pensamos otras formas de resolverlo. Estas son las que descartamos y por qué.

Una sola clase Planta con un atributo “tipo”

Era lo más fácil: una clase con todos los atributos y un campo que diga si es solar, eólica, hidráulica o térmica. El problema es que cada planta se quedaría con un montón de atributos en null que no le corresponden, el IREP dependería del tipo y cumpleCuota terminaría siendo un switch gigante. Encima, cada tipo nuevo nos obligaba cambiar una clase que ya funciona. Y tampoco usa herencia ni polimorfismo, que es justo lo que se solicita en el TP.

Las cuatro plantas heredando directo de Planta

Así también se puede resolver, pero la fuente primaria y la eficiencia promedio quedarían repetidas en tres clases, igual que sus IREP. Además, para contar las renovables por fuente habría que

preguntar tipo por tipo, y si mañana aparece otra renovable habría que acordarse de copiar todo de nuevo. Con PlantaRenovable eso está en un solo lugar.

Un solo Monitoreo con campos opcionales

Habíamos pensado en un único monitoreo con CO2, combustible y velocidad del viento como campos que a veces se usan y a veces no. Lo malo es que un monitoreo de una planta solar tendría campos vacíos, y nada impediría cargarle velocidad de viento a una térmica o dejar sin CO2 un monitoreo térmico. Con subclases, cada monitoreo tiene justo los datos que le pertenecen. Para la solar y la hidráulica, que no tienen datos propios, alcanza con el Monitoreo común.

Dónde guardar los monitoreos

En la primera parte elegimos que cada planta guardara sus registros, porque casi todas las consultas trabajan con una sola planta y así la unicidad por fecha y hora salía de la clave. La descartamos al implementar porque los nuevos métodos trabajan sobre el conjunto completo, y con una lista única alcanza un solo recorrido con Iterator. El precio es que chequear que no haya otro monitoreo en la misma hora implica recorrer la lista, y que el histograma recorre los monitoreos de todas las plantas (ver el análisis de complejidad).

Listas o diccionarios para las plantas

Con una lista de plantas, buscar una por nombre es recorrer toda la lista, y para evitar nombres repetidos hay que revisarlos uno por uno. Con un diccionario, las búsquedas son directas con la clave, que es lo que necesitamos para el O(1) de activar y desactivar. Por eso usamos un HashMap, y conservamos además la LinkedList para respetar el orden en que se registraron las plantas.

Dos colecciones, una de plantas activas y otra de inactivas

Para saber rápido cuáles están activas se nos ocurrió tener dos colecciones y pasar la planta de una a otra cuando cambia el estado. Pero es el doble de laburo, tenemos que mantener las dos al día y se complica asegurar que no haya nombres repetidos entre ambas. Con el estado como un booleano en la planta, más su historial de inactividad, alcanza con cambiar ese valor, y si hace falta filtrar por estado se recorre la colección una vez.

Un objeto aparte para calcular la cuota (Strategy)

Evaluamos que cada planta tuviera un objeto “calculador de cuota” intercambiable, en lugar de redefinir el método. Calcular la cuota quedaba resuelto, pero los datos propios de cada tipo igual necesitan una subclase, así que tendríamos dos jerarquías en paralelo para lo mismo. Redefinir cuotaMinima() y cumpleCuota() en cada subclase es más simple y aprovechamos a fondo el polimorfismo.